Introduction to Loops: For Loop and While Loop

For Loop

1. For Loop Incremental Type with Comparison Operators and Plus-Based Incrementing

The for loop is used when you know the exact number of iterations. The basic syntax includes initialization, condition, and increment/decrement.

Syntax:

```java
for (initialization; condition; increment) {
    // code to be executed
}
```

Examples:

```java
// Example 1: Print numbers from 1 to 5
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
// Output: 1, 2, 3, 4, 5
```

```java
// Example 2: Print even numbers from 2 to 10
for (int i = 2; i <= 10; i += 2) {
    System.out.println(i);
}
// Output: 2, 4, 6, 8, 10
```

```java
// Example 3: Using different comparison operators
for (int i = 0; i < 5; i++) {
    System.out.println("Value: " + i);
}
// Output: Value: 0, Value: 1, Value: 2, Value: 3, Value: 4
```

2. For Loop Decremental Type with Comparison Operators and Minus-Based Decrementing

For loops can also count backwards using decrement operators.

Examples:

```java
// Example 1: Print numbers from 5 to 1
for (int i = 5; i >= 1; i--) {
    System.out.println(i);
}
// Output: 5, 4, 3, 2, 1
```

```java
// Example 2: Countdown from 10 to 0 by 2
for (int i = 10; i >= 0; i -= 2) {
    System.out.println(i);
}
// Output: 10, 8, 6, 4, 2, 0
```

```java
// Example 3: Using greater than comparison
for (int i = 8; i > 2; i--) {
    System.out.println("Count: " + i);
}
// Output: Count: 8, Count: 7, Count: 6, Count: 5, Count: 4, Count: 3
```

3. Char Data Type as Initialization in For Loop

You can use char data type as the loop variable to iterate through characters.

Examples:

```java
// Example 1: Print letters from 'a' to 'e'
for (char c = 'a'; c <= 'e'; c++) {
    System.out.println(c);
}
// Output: a, b, c, d, e
```

```java
// Example 2: Print uppercase letters from 'A' to 'F'
for (char letter = 'A'; letter <= 'F'; letter++) {
    System.out.println("Letter: " + letter);
}
// Output: Letter: A, Letter: B, Letter: C, Letter: D, Letter: E, Letter: F
```

```java
// Example 3: Reverse alphabet from 'z' to 'v'
for (char c = 'z'; c >= 'v'; c--) {
    System.out.print(c + " ");
}
// Output: z y x w v
```

4. Nested For Loop

A nested for loop is a loop inside another loop. The inner loop completes all its iterations for each iteration of the outer loop.

Examples:

```java
// Example 1: Print a multiplication table
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        System.out.print(i * j + " ");
    }
    System.out.println();
}
// Output:
// 1 2 3
// 2 4 6
// 3 6 9
```

```java
// Example 2: Print a pattern of stars
for (int i = 1; i <= 4; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("* ");
    }
    System.out.println();
}
// Output:
// *
// * *
// * * *
// * * * *
```

```java
// Example 3: Print coordinates
for (int x = 0; x < 3; x++) {
    for (int y = 0; y < 3; y++) {
        System.out.println("(" + x + "," + y + ")");
    }
}
// Output: (0,0), (0,1), (0,2), (1,0), (1,1), (1,2), (2,0), (2,1), (2,2)
```

While Loop

1. While Loop Incremental Type with Comparison Operators and Plus-Based Incrementing

The while loop continues executing as long as the condition is true. You need to manually handle initialization and increment/decrement.

Syntax:

```java
initialization;
while (condition) {
    // code to be executed
    increment/decrement;
}
```

Examples:

```java
// Example 1: Print numbers from 1 to 5
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}
// Output: 1, 2, 3, 4, 5
```

```java
// Example 2: Sum of first 4 numbers
int num = 1;
int sum = 0;
while (num <= 4) {
    sum += num;
    num++;
}
System.out.println("Sum: " + sum);
// Output: Sum: 10
```

```java
// Example 3: Print even numbers from 2 to 8
int count = 2;
while (count < 9) {
    System.out.println(count);
    count += 2;
}
// Output: 2, 4, 6, 8
```

2. While Loop Decremental Type with Comparison Operators and Minus-Based Decrementing

While loops can also count backwards using decrement operations.

Examples:

```java
// Example 1: Countdown from 5 to 1
int i = 5;
while (i >= 1) {
    System.out.println(i);
    i--;
}
// Output: 5, 4, 3, 2, 1
```

```java
// Example 2: Print numbers from 10 to 6
int num = 10;
while (num > 5) {
    System.out.println("Number: " + num);
    num--;
}
// Output: Number: 10, Number: 9, Number: 8, Number: 7, Number: 6
```

```java
// Example 3: Countdown by 3
int counter = 15;
while (counter >= 3) {
    System.out.print(counter + " ");
    counter -= 3;
}
// Output: 15 12 9 6 3
```

3. Char Data Type as Initialization in While Loop

You can use char data type as the loop variable in while loops too.

Examples:

```java
// Example 1: Print letters from 'a' to 'd'
char c = 'a';
while (c <= 'd') {
    System.out.println(c);
    c++;
}
// Output: a, b, c, d
```

```java
// Example 2: Print uppercase letters backwards
char letter = 'E';
while (letter >= 'A') {
    System.out.print(letter + " ");
    letter--;
}
// Output: E D C B A
```

```java
// Example 3: Print vowels
char vowel = 'a';
while (vowel <= 'u') {
    if (vowel == 'a' || vowel == 'e' || vowel == 'i' || vowel == 'o' || vowel == 'u') {
        System.out.print(vowel + " ");
    }
    vowel++;
}
// Output: a e i o u
```

4. Nested While Loop

A nested while loop contains one while loop inside another while loop.

Examples:

java

```java
// Example 1: Print a 3x3 grid of numbers
int i = 1;
while (i <= 3) {
    int j = 1;
    while (j <= 3) {
        System.out.print(i + "" + j + " ");
        j++;
    }
    System.out.println();
    i++;
}
// Output:
// 11 12 13
// 21 22 23
// 31 32 33
```

java

```java
// Example 2: Print a pattern
int row = 1;
while (row <= 3) {
    int col = 1;
    while (col <= row) {
        System.out.print("# ");
        col++;
    }
    System.out.println();
    row++;
}
// Output:
// #
// # #
// # # #
```

```java
// Example 3: Multiplication table using nested while
int outer = 1;
while (outer <= 2) {
    int inner = 1;
    while (inner <= 3) {
        System.out.print((outer * inner) + " ");
        inner++;
    }
    System.out.println();
    outer++;
}
// Output:
// 1 2 3
// 2 4 6
```

Quiz Questions

Question 1: What will be the output of the following code?

```java
for (int i = 2; i < 6; i += 2) {
    System.out.print(i + " ");
}
```

a) 2 4 6
b) 2 4
c) 2 3 4 5
d) 4 6

Answer: b) 2 4

Question 2: Which of the following is the correct syntax for a while loop? a) while (condition) { code; increment; }
b) while condition { code; }
c) while (condition); { code; }
d) while { condition } code;

Answer: a) while (condition) { code; increment; }

Question 3: What will be the output of this nested for loop?

```java
for (int i = 1; i <= 2; i++) {
    for (int j = 1; j <= 2; j++) {
        System.out.print(i + j + " ");
    }
}
```

a) 1 2 3 4

b) 2 3 3 4

c) 1 1 2 2

d) 2 3 4 5

Answer: b) 2 3 3 4

Question 4: What happens if you forget to increment the counter in a while loop? a) The program will crash

b) The loop will run only once

c) The loop will become infinite

d) The program will skip the loop

Answer: c) The loop will become infinite

Question 5: Which loop is better when you know the exact number of iterations? a) while loop

b) for loop

c) Both are equally good

d) Neither should be used

Answer: b) for loop

Practice Coding Questions

Question 1: Write a for loop that prints all odd numbers from 1 to 15.

Question 2: Create a while loop that calculates the factorial of a number 5.

Question 3: Write a nested for loop that prints the following pattern:

```
1
12
123
1234
```

Question 4: Use a for loop with char data type to print all uppercase letters from 'Z' to 'U' in reverse order.

Question 5: Write a nested while loop that prints a 4x4 multiplication table (1x1 to 4x4).